

Literature Review:

Paper Title	Method	Dataset	Advantages	Limitations	Research Gap
Towards Secure Cloud-Native Computing: Unveiling Kubernetes Misconfigurations with Large Language Models	Builds a structured taxonomy of Kubernetes misconfiguration types and empirically benchmarks rule-based scanners against LLM-based detectors across that taxonomy.	Curated corpus of real-world Kubernetes configuration files (KCFs), compared against rule-based tools (Checkov, KubeLinter, Terrascan).	Provides a reusable, structured misconfiguration taxonomy; empirically shows LLMs catch a broader range of issues than static rule-based tools.	Relies on general-purpose commercial LLMs, raising cost, privacy, and consistency concerns; detection-only, with no automated repair or retrieval grounding.	Detection-only, single-platform (Kubernetes) taxonomies like this remain disconnected from a coordinated multi-agent pipeline that also validates deployment, security, and runtime drift across Terraform, CloudFormation, and CDK — the integration this project's IaC-parsing and security-assessment agents are designed to provide.
LLMSecConfig: An LLM-Based Approach for Fixing Software Container Misconfigurations	Combines static analysis tools (SATs) with LLMs through advanced prompting and Retrieval-Augmented Generation (RAG) in an iterative detect-retrieve-patch-revalidate loop.	1,000 real-world Kubernetes configuration files.	Achieves a 94% repair success rate with minimal newly introduced issues; demonstrates RAG's clear advantage over a plain-LLM baseline (~40% success).	Scope limited to container-orchestrator configurations rather than the broader multi-cloud IaC ecosystem (Terraform/CloudFormation); repair quality depends on the underlying SAT's coverage.	Reactive, post-deployment repair scoped to containers/Kubernetes does not extend to the pre-deployment, multi-format (Terraform/CloudFormation/CDK) validation and proactive security assessment this project's agents must perform before infrastructure reaches smart-manufacturing cloud environments.
ARPaCCino: An Agentic-RAG for Policy as Code Compliance	An agentic system combining LLMs, RAG, and tool-based validation to convert natural-language policy into formal Rego rules, assess IaC compliance, and iteratively refine IaC.	Natural-language policy descriptions paired with target IaC configurations across several IaC technologies, evaluated via OPA compliance checks across three LLM backends.	Agentic-RAG substantially outperforms both plain LLM and plain RAG; modular design extends to niche or emerging IaC frameworks via added knowledge bases.	Depends on the completeness of the supplied knowledge base per framework; iterative agentic loops raise latency/cost; correctness remains probabilistic, not formally verified.	Policy-as-code compliance checking here depends on manually curated, framework-specific knowledge bases and stops at compliance; it does not extend to deployment verification, configuration-drift correlation with live telemetry, or cost analysis, all of which the proposed multi-agent framework must combine into one coordinated workflow.

Paper Title	Method	Dataset	Advantages	Limitations	Research Gap
Kubernetes Misconfigurations in the Wild: Taxonomy, Evolution, and Automated Repair with Large Language Models	Uses multiple LLMs to construct a domain-specific taxonomy via a five-step process, studies how misconfiguration patterns evolve over time, and proposes a hybrid LLM plus deterministic schema-enforcement repair method.	Large real-world corpus of Kubernetes manifests mined "in the wild."	Achieves about 98.5% correction accuracy; longitudinal analysis shows how misconfiguration patterns evolve, informing which rules become outdated.	Still Kubernetes-specific rather than covering the wider IaC landscape; taxonomy construction relies on LLM judgment, risking categorization inconsistency across models.	Being Kubernetes-only and retrospective, this taxonomy cannot inform real-time, multi-format validation and has no link to live cloud state, leaving a gap for a framework that correlates static misconfiguration patterns with runtime telemetry across the broader AWS service set this project covers.
LLM-Based Misconfiguration Detection for AWS Serverless Computing	Zero-shot prompting framework (SIsDetector) using multi-dimensional constraints (resource types, entries, values, dependencies) and chain-of-thought reasoning for static misconfiguration detection.	Curated set of 110 serverless (AWS SAM) configuration files, spanning correct, real-world misconfigured, and injected-error cases.	Achieves 79.75% F1, outperforming prior data-driven baselines; generalizes well across multiple LLM backends (GPT-4o, Llama 3.1, Gemini 1.5 Pro, DeepSeek V3).	Zero-shot detection only, with no retrieval grounding or automated repair; small evaluation set; scope limited to a single cloud provider's serverless schema.	Zero-shot, detection-only coverage of a single format (AWS SAM) on a small dataset (110 files) does not generalize to the multi-format (Terraform, CloudFormation, CDK, Kubernetes, SAM), multi-agent validation and repair pipeline with runtime telemetry integration that this project proposes.